My RAG Chatbot on AWS

Kaelem Turay | October

What I built

I built a chatbot that answers questions using only my own documents. If the answer is in the documents, it tells me and says which file it came from. If it isn't, it says it doesn't know instead of making something up. This kind of setup is called RAG, which stands for retrieval-augmented generation.

To test it, I made up a small company with five documents: a refund policy, a shipping policy, a time-off policy, a tent product FAQ, and a support guide. Since I knew what was in them, I could tell right away if an answer was wrong.

How it works

- My documents sit in an Amazon S3 bucket.
- An Amazon Bedrock Knowledge Base splits them into pieces and stores them so they can be searched.
- When I ask a question, a small function on AWS Lambda finds the best matching pieces and sends them, along with my question, to a language model.
- The model writes a short answer from those pieces and names the source file.
- A simple web page I made shows the chat.

What I used

- Amazon S3 for the documents
- Amazon Bedrock Knowledge Bases (Titan Text Embeddings V2 for the search part)
- Amazon Nova Pro for writing the answers
- AWS Lambda for the backend, written in Node.js
- One HTML file for the chat page

Problems I ran into

- **The console test wouldn't generate answers.** Searching the documents worked, but the built-in answer step gave an error about prompt templates. I fixed it by doing the search and the answer in my own code instead.
- **I couldn't use Claude from code.** Every Claude model gave an access error on my account, even though it worked in the playground. I switched to Amazon Nova Pro, which worked right away. Changing models later is a one-line edit.

- **The first version of the backend gave a 502 error.** Lambda web functions need a small web server, not the usual handler function. I rewrote it as a server.
- **It couldn't find the AWS software packages.** The web function doesn't include them, so I installed them and deployed the whole folder with the AWS command line tool.

How I tested it

I asked it questions I already knew the answers to. It correctly said I have 45 days to return an item. When I asked what happens if a tent arrives damaged, it pulled the right details from the refund policy. When I asked for the CEO's name, which isn't in any document, it said it didn't have that information. It still listed a source file for that answer, so I tightened the instructions I give the model.

Things to be careful about

- The endpoint has no login yet, so anyone with the link could use it and run up my bill. I'm keeping the link private and have a budget alert set.
- When I'm done experimenting, I'll delete the Knowledge Base so it doesn't keep costing money.

What I'd do next

- Add a login so only approved people can use it.
- Make it update automatically when I change the documents.
- Try Claude again once the access problem is sorted out and compare the answers.